Optimizing Matrix Multiplication

Created and Presented by Kayle Megginson
CS 351: Analysis of Algorithms

Introduction

The Problem of Sparse Matrix Multiplication

- Sparse matrices contain many zero elements, making traditional storage and operations inefficient.

- Challenges: Large-scale datasets, high computational cost, and memory inefficiencies in dense representations.

- **Solution Overview:** Use the Compressed Sparse Row (CSR) format to optimize storage and implement efficient multiplication algorithms

- **Approach:** Design and implement the `SparseMatrix` class with CSR-based multiplication, ensuring optimized runtime and space usage.

Background and Research

Key Algorithms and Data Structures

- **Compressed Sparse Row (CSR):**
  - Stores only non-zero elements, their column indices, and row pointers.
  - Significantly reduces memory usage compared to dense matrices.
- **Research Insights:**
  - CSR is ideal for sparse matrices where zero elements dominate.
  - Trade-offs: Complexity in matrix construction vs. reduced runtime and memory usage during operations.
- **Challenges Addressed:** Memory overhead in dense representations and inefficiencies in direct element-wise operations.

- **Strassen's Algorithm:**
  - Uses recursion to calculate product entries
  - Theoretical Algorithmic Complexity of O(n^2.81)

Standard Matrix Multiplication

$$\begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix} \begin{bmatrix} b_1 & b_2 & b_3 \\ b_4 & b_5 & b_6 \\ b_7 & b_8 & b_9 \end{bmatrix} = \begin{bmatrix} c_1 & c_2 & c_3 \\ c_4 & c_5 & c_6 \\ c_7 & c_8 & c_9 \end{bmatrix}$$

```python
import numpy as np

def matrix_multiplication(A, B):
    # Get the dimensions of the matrices
    rows_A, cols_A = A.shape
    rows_B, cols_B = B.shape

    # Check if the matrices can be multiplied
    if cols_A != rows_B:
        raise ValueError("Number of columns in A must be equal to number of rows in B")

    # Create the result matrix with zeros
    result = np.zeros((rows_A, cols_B))

    # Perform the matrix multiplication
    for i in range(rows_A):
        for j in range(cols_B):
            for k in range(cols_A):
                result[i, j] += A[i, k] * B[k, j]

    # Convert to integer if the result is a whole number
    if np.all(result == result.astype(int)):
        result = result.astype(int)

    return result
```

Strassen's Algorithm

$$\underset{A}{\begin{bmatrix} a & b \\ c & d \end{bmatrix}} \times \underset{B}{\begin{bmatrix} e & f \\ g & h \end{bmatrix}} = \underset{C}{\begin{bmatrix} ae + bg & af + bh \\ ce + dg & cf + dh \end{bmatrix}}$$

A, B and C are square metrices of size N x N
a, b, c and d are submatrices of A, of size N/2 x N/2
e, f, g and h are submatrices of B, of size N/2 x N/2

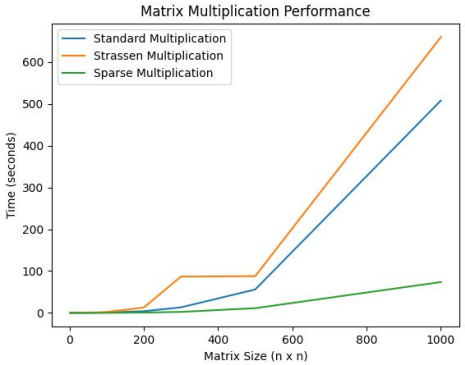
p1 = a(f - h)
p3 = (c + d)e
p5 = (a + d)(e + h)
p7 = (a - c)(e + f)

p2 = (a + b)h
p4 = d(g - e)
p6 = (b - d)(g + h)

The A x B can be calculated using above seven multiplications.
Following are values of four sub-matrices of result C

$$\underset{A}{\begin{bmatrix} a & b \\ c & d \end{bmatrix}} \times \underset{B}{\begin{bmatrix} e & f \\ g & h \end{bmatrix}} = \underset{C}{\begin{bmatrix} p5 + p4 - p2 + p6 & p1 + p2 \\ p3 + p4 & p1 + p5 - p3 - p7 \end{bmatrix}}$$

p1, p2, p3, p4, p5, p6 and p7 are submatrices of size N/2 x N/2




```python
import numpy as np

def add_matrix(A, B):
    return [[A[i][j] + B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def subtract_matrix(A, B):
    return [[A[i][j] - B[i][j] for j in range(len(A[0]))] for i in range(len(A))]

def strassen_multiply(A, B):
    n = len(A)
    if n == 1:
        return [[A[0][0] * B[0][0]]]
    else:
        mid = n // 2

        A11 = [row[:mid] for row in A[:mid]]
        A12 = [row[mid:] for row in A[:mid]]
        A21 = [row[:mid] for row in A[mid:]]
        A22 = [row[mid:] for row in A[mid:]]

        B11 = [row[:mid] for row in B[:mid]]
        B12 = [row[mid:] for row in B[:mid]]
        B21 = [row[:mid] for row in B[mid:]]
        B22 = [row[mid:] for row in B[mid:]]

        M1 = strassen_multiply(add_matrix(A11, A22), add_matrix(B11, B22))
        M2 = strassen_multiply(add_matrix(A21, A22), B11)
        M3 = strassen_multiply(A11, subtract_matrix(B12, B22))
        M4 = strassen_multiply(A22, subtract_matrix(B21, B11))
        M5 = strassen_multiply(add_matrix(A11, A12), B22)
        M6 = strassen_multiply(subtract_matrix(A21, A11), add_matrix(B11, B12))
        M7 = strassen_multiply(subtract_matrix(A12, A22), add_matrix(B21, B22))

        C11 = add_matrix(subtract_matrix(add_matrix(M1, M4), M5), M7)
        C12 = add_matrix(M3, M5)
        C21 = add_matrix(M2, M4)
        C22 = add_matrix(subtract_matrix(add_matrix(M1, M3), M2), M6)

        C = []
        for i in range(mid):
            C.append(C11[i] + C12[i])
        for i in range(mid):
            C.append(C21[i] + C22[i])

        return C
```

Design Overview

Designing the `SparseMatrix` Class

- **Key Components:**
  - `values`: Non-zero elements.
  - `col_indices`: Column indices of the non-zero elements.
  - `row_pointers`: Start and end indices of each row in `values`.

- **Core Operations:**
  - **Matrix Initialization**: Efficiently handle non-zero value input.
  - **Matrix Multiplication**: Multiply only the relevant rows/columns to avoid unnecessary computations.
  - **Conversion to Dense**: Enable visualization of the matrix for debugging and validation.

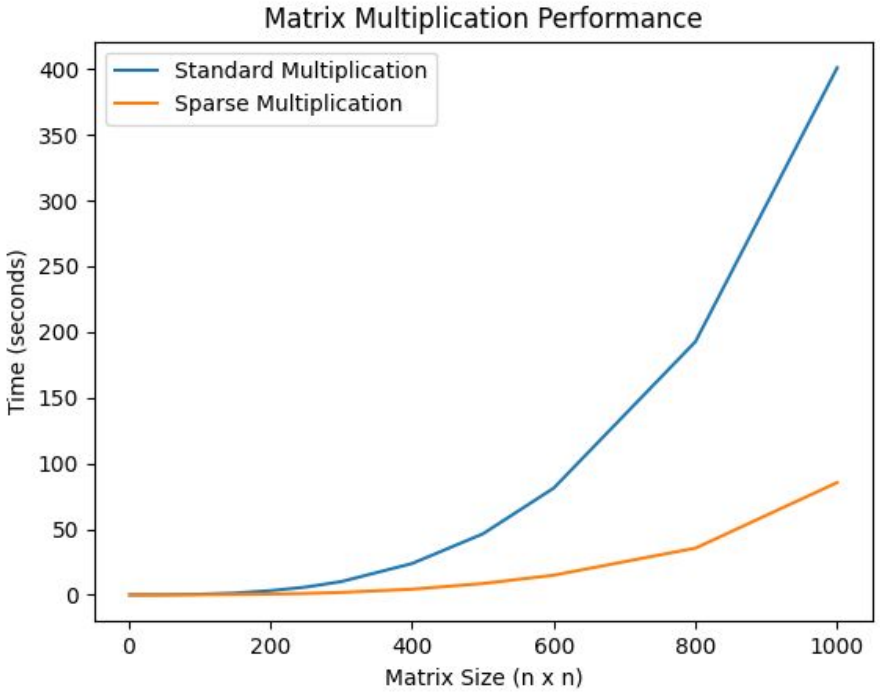
```python
class SparseMatrix:
    def __init__(self, rows, cols):
        self.rows = rows
        self.cols = cols
        self.values = []  # List to store non-zero values
        self.col_indices = []  # List to store column indices of the non-zero values
        self.row_pointers = [0]  # List to store the start index of each row in the values list

    def add_value(self, row, col, value):
        if value != 0:
            self.values.append(value)  # Add the non-zero value
            self.col_indices.append(col)  # Add the column index of the value
            if len(self.row_pointers) <= row + 1:
                # Extend row_pointers list if necessary
                self.row_pointers.extend([self.row_pointers[-1]] * (row + 1 - len(self.row_pointers) + 1))
            self.row_pointers[row + 1] += 1  # Increment the pointer for the next row

    def multiply(self, other):
        if self.cols != other.rows:
            raise ValueError("Matrix dimensions do not match for multiplication")

        result = SparseMatrix(self.rows, other.cols)  # Initialize the result matrix

        for i in range(self.rows):
            row_start = self.row_pointers[i]
            row_end = self.row_pointers[i + 1]
            row_values = self.values[row_start:row_end]
            row_indices = self.col_indices[row_start:row_end]

            result_row = [0] * other.cols  # Initialize the result row
            for j, value in zip(row_indices, row_values):
                other_row_start = other.row_pointers[j]
                other_row_end = other.row_pointers[j + 1]
                other_row_values = other.values[other_row_start:other_row_end]
                other_row_indices = other.col_indices[other_row_start:other_row_end]

                for k, other_value in zip(other_row_indices, other_row_values):
                    result_row[k] += value * other_value  # Perform multiplication and add to result row

            for col, value in enumerate(result_row):
                if value != 0:
                    result.add_value(i, col, value)  # Add non-zero values to the result matrix

        return result

    def to_dense(self):
        dense_matrix = [[0] * self.cols for _ in range(self.rows)]  # Initialize a dense matrix with zeros
        for i in range(self.rows):
            row_start = self.row_pointers[i]
            row_end = self.row_pointers[i + 1]
            for j in range(row_start, row_end):
                col = self.col_indices[j]
                dense_matrix[i][col] = self.values[j]  # Fill the dense matrix with non-zero values
        return dense_matrix
```

Analysis and Results

Key Observations:

- CSR format drastically reduces memory usage compared to dense matrices.

- Multiplication performance scales well with sparsity.



Reflection

Lessons Learned

- **Challenges Faced:**
  - Handling dynamic updates to CSR during matrix construction.
  - Debugging edge cases like mismatched dimensions.
- **Improvements Over Time:**
  - Optimized `multiply` for reduced intermediate memory usage.
  - Enhanced readability and maintainability of the code.

And also Strassen's algorithm is 🗑️

Uses of Matrix Multiplication:

- solving systems of linear equations
- modeling complex systems
- analyzing transformations and representing relationships between multiple variables, including applications like image processing, data analysis, network analysis, and population modeling

Conclusion

Success, Limitations, and Future Work

- **Summary:**
  - Successfully implemented CSR-based sparse matrix multiplication.
  - Achieved significant optimizations in memory and runtime.
- **Limitations:**
  - Overhead in constructing the CSR representation.
  - Limited scalability for parallelized operations.

Thank you!!